

Syllabus: Programmazione e Tecnologie Web

Giuseppe Capasso

Tipologia: Corso ITS

Materia: Programmazione e Tecnologie Web

Durata totale: 35 ore (7 lezioni da 5 ore)

Obiettivo del corso: Fornire una panoramica completa sul funzionamento del web moderno, rendendo gli studenti capaci di sviluppare e distribuire applicazioni web end-to-end, comprendendo architetture, protocolli e interfacce.

Prerequisiti

- Concetti base sulle reti di calcolatori (indirizzi IP, porte, concetto di rete).
- Logica di programmazione di base (variabili, cicli, condizioni, funzioni).

Struttura del Corso

Lezione 1: Fondamenti del Web e Setup dell'Ambiente

Obiettivo: Comprendere come viaggiano i dati sul web, configurare gli strumenti di lavoro e scrivere le prime righe di codice frontend.

- **Teoria (2 ore):** Il modello architetturale Client-Server: ruoli del Frontend e del Backend. Il protocollo HTTP: cos'è, come funziona (Request/Response), concetti di URL e DNS.
- **Setup e Strumenti (1 ora):** Installazione e configurazione dell'ambiente: Visual Studio Code. Controllo di versione: concetti base di Git. Uso consapevole degli assistenti AI per la programmazione.
- **Laboratorio Pratico (2 ore):** Introduzione pratica a HTML (struttura semantica), CSS (selettori e styling base) e JavaScript (interazione col DOM). *Esercizio:* Creazione di una semplice pagina web interattiva.

Lezione 2: Il Web Statico vs Dinamico e la Logica Client-Side

Obiettivo: Differenziare le tipologie di siti web e imparare a gestire lo stato e i dati direttamente nel browser.

- **Teoria (1.5 ore):** Siti statici vs Siti dinamici (Server-Side Rendering vs Client-Side Rendering): Introduzione ai generatori di siti statici (SSG).
- **Laboratorio Pratico - Parte 1 (1 ore):** Generazione di un sito web statico utilizzando **Hugo**. Come strutturare i contenuti e compilare il sito.
- **Laboratorio Pratico - Parte 2 (2.5 ore):** *Esercizio guidato:* Creazione di una Todo App "Client-Only" (HTML/CSS/JS). Implementazione del paradigma CRUD (Create, Read, Update, Delete) utilizzando il DOM e salvando i dati nel `localStorage` del browser.

Lezione 3: Il Backend e le API RESTful

Obiettivo: Spostare il focus sul server, imparando a far comunicare sistemi diversi e comprendendo i concetti di sicurezza base.

- **Teoria (2 ore):** Cosa significa "fare backend". Le API (Application Programming Interface): cosa sono e perché sono il collante del web. Principi delle REST API: Metodi HTTP (GET, POST, PUT, DELETE), status code, e formato JSON. Autenticazione: Basic, Bearer Token (JWT), OAuth.

- **Laboratorio Pratico (3 ore):** Utilizzo di **Postman** per esplorare e testare API esistenti. Utilizzo di Python per effettuare richieste HTTP (libreria **requests**). *Esercizio:* Interazione con le API di **Webex**. Creazione di un semplice script Python che invia messaggi tramite un Bot Webex sfruttando un Bearer Token.

Lezione 4: Sviluppare il proprio Backend

Obiettivo: Scrivere un server personalizzato, esporre le proprie API e connetterle al frontend precedentemente creato.

- **Teoria (1 ora):** Architettura di un server web in Python (Flask) e JavaScript (Express.js). Cenni sui framework moderni e architetture enterprise (es. FastAPI, Django, Spring Boot).
- **Laboratorio Pratico (4 ore):** Setup di un server backend semplice in Python (Flask) o Node.js (Express) per gestire i Todo. Implementazione dei selettori API REST per il CRUD dei task. Aggiunta di un sistema di autenticazione base (es. verifica di un token statico o basic auth) per proteggere le rotte. *Integrazione:* Trasformazione della Todo App della Lezione 2 in un'applicazione Client-Server. Il frontend ora chiama le API del nostro backend tramite `fetch()`.

Lezione 5: I Framework Frontend Moderni (CSR vs SSR)

Obiettivo: Capire perché e come si usano i framework JavaScript moderni per gestire applicazioni complesse.

- **Teoria (1.5 ore):** L'evoluzione del frontend: perché jQuery non bastava più. Il problema della manipolazione del DOM. Il concetto di Single Page Application (SPA) e Componenti. Panoramica su React, Angular e Vue.
- **Laboratorio Pratico (3.5 ore):** Introduzione a React e al framework **Next.js** (fondamenti). Gestione dello stato in React (`useState`, `useEffect`). *Esercizio:* Riscrittura del frontend della Todo App utilizzando React/Next.js e connessione al backend (sviluppato nella lezione 4).

Lezione 6: Il Web Real-Time e Multimediale

Obiettivo: Superare il paradigma request/response esplorando le tecnologie per la comunicazione in tempo reale.

- **Teoria (1.5 ore):** Limiti dell'HTTP standard per il real-time (polling vs connessioni persistenti). Il protocollo **WebSocket**: comunicazione bidirezionale full-duplex. Cenni teorici su **WebRTC** (Web Real-Time Communication) per lo streaming audio/video peer-to-peer.
- **Laboratorio Pratico (3.5 ore):** Implementazione di un server WebSocket (es. con Socket.io o librerie standard Python/Node). *Esercizio:* Costruzione di una piccola applicazione di chat testuale di gruppo in tempo reale. Integrazione base di flussi video utilizzando le API HTML5 della fotocamera/microfono.

Lezione 7: Il Futuro del Web e Verifica Finale

Obiettivo: Concludere il corso con uno sguardo alle tecnologie emergenti e valutare le competenze acquisite.

- **Teoria (4 ore):** L'evoluzione dei protocolli: problemi di HTTP/1.1. Caratteristiche e vantaggi di **HTTP/2** (multiplexing, server push). Introduzione a **HTTP/3** e al protocollo **QUIC** (basato su UDP): come riducono la latenza e migliorano le performance su reti mobili. Alternative a JavaScript: WebAssembly (WASM).
- **Test Finale (1 ora):** Somministrazione test finale.